

AI-Based Urban Traffic Congestion Prediction System

Project Report | College / Internship Submission

Project Title	AI-Based Urban Traffic Congestion Prediction System
Student Name	_____
Registration / Roll No.	_____
Programme / Branch	_____
Institution / College	_____
Project Guide / Supervisor	_____
Submission Year	20____
Project Status	Stages 1-8 complete; 451 automated tests passing

Prepared by

Name: _____ Roll No.: _____

This report documents a verified software build (Stages 1-8). All figures are execution results from the repository; no accuracy claims for real-world traffic are made.

The enclosed forecasts are demonstrative until the model is retrained and validated against real-world ground truth.

Abstract

Urban traffic congestion is discovered too late: commuters learn about it once they are inside it, and traffic management reacts after congestion has already formed. This project builds a modular AI-based system that ingests urban traffic observations, engineers leakage-safe features, trains a machine-learning model to predict congestion level per road segment, and serves current state and near-horizon forecasts through a documented REST API and an interactive map dashboard.

The implementation spans eight stages that are each verified end to end: a six-stage data pipeline, a Random Forest classifier (version 4.0.0, 26 features) evaluated against an honest majority-class baseline, PostgreSQL persistence with Alembic migrations, a real-time ingestion path through the TomTom traffic API and the Open-Meteo weather API behind a provider interface, a background collection scheduler, a dashboard aggregation API, a static front-end dashboard, and a final hardening pass (production configuration guard, security review, deployment guide and this report). The complete build is verified by 451 automated tests.

Honesty is treated as a first-class requirement: the dataset used to train the model is a clearly labelled simulated fixture, so every forecast is demonstrative rather than field-validated. Connecting a live traffic provider does not change that. The system is designed so the same, tested pipeline can be retrained and revalidated on real observations once sufficient history is collected.

Problem Statement

Urban road networks experience congestion that people only discover once they are already inside it. Reactive traffic control responds after congestion has formed, and static timetables cannot account for the day-to-day variation caused by incidents, weather, construction and shifting peak hours. The consequences are measurable: lost travel time, wasted fuel, avoidable emissions, and unreliable travel planning.

Existing dashboards mostly describe current conditions. They answer 'where is it congested right now?' but not 'will this road be congested in 30 minutes?' - which is the question that would actually let someone change their behaviour. Additionally, congestion labels are usually produced by simple threshold rules; a severe-but-rare class is easily ignored by an accuracy-optimising approach, which is why this project evaluates with macro F1 against a majority-class baseline.

Objectives

- Collect urban traffic observations into a persistent store, from a live provider when a traffic API key is configured and from a clearly labelled simulation otherwise.
- Engineer leakage-safe features from time, road geometry, traffic flow and weather.
- Train and evaluate a machine-learning classifier against a honest majority-class baseline on a chronological split.
- Expose predictions through a documented REST API with a uniform error envelope.
- Visualise current and predicted congestion on an interactive map dashboard.
- Harden the result for rollout: production configuration cannot silently fall back to simulated data, and everything is verifiable by automated tests.

Existing System

The context this project addresses is served today by systems that are largely descriptive and reactive:

- Live condition dashboards and traffic apps report current congestion, not near-term forecasts.
- Traffic control reacts after congestion appears; there is no forward-looking trigger.
- Static timetable and routing systems cannot incorporate incidents, weather or peak-hour variation.
- Many congestion displays colour by hard-coded threshold rules and do not state how a label was derived or what data it was computed from.

A further limitation relevant to any data-driven replacement is that no self-serve traffic API publishes vehicle throughput, so any system that pretends to know traffic volume is fabricating it. The proposed system stores 'unknown' where an upstream source does not publish a value.

Proposed System

The proposed system is a modular, end-to-end pipeline with five standalone parts:

- Data ingestion: a provider interface with a labelled simulated provider and a live provider (TomTom Flow Segment Data + Open-Meteo weather), stored with source provenance on every row.
- Data pipeline and model: a six-stage, leakage-controlled preprocessing pipeline feeding a Random Forest classifier (v4.0.0) whose artifacts carry machine-readable provenance.
- Prediction engine: after a reading is stored it is scored against that segment's history and the forecast is persisted; insufficient history is an explicit error, never an invented prediction.
- Dashboard API + frontend: five aggregation endpoints (current, trends, locations, status, model) consumed by a static dashboard with a Leaflet map and Chart.js analytics.
- Hardened operation: a production configuration guard, one error envelope, upstream-failure classification, and full deployment documentation.

Key design decisions: the ML layer never imports the web layer; routes stay thin while logic lives in services and all SQL in repositories; provenance is a first-class column; an unavailable capability is an explicit error (e.g. 422 insufficient_history, 503 model_not_available, 503 provider_not_configured); an unmeasured value is stored NULL, never estimated.

System Architecture

The architecture separates the browser, the FastAPI service layer, the independent ML pipeline, the PostgreSQL store and the external data providers. The background collection scheduler is opt-in and never blocks the request path.

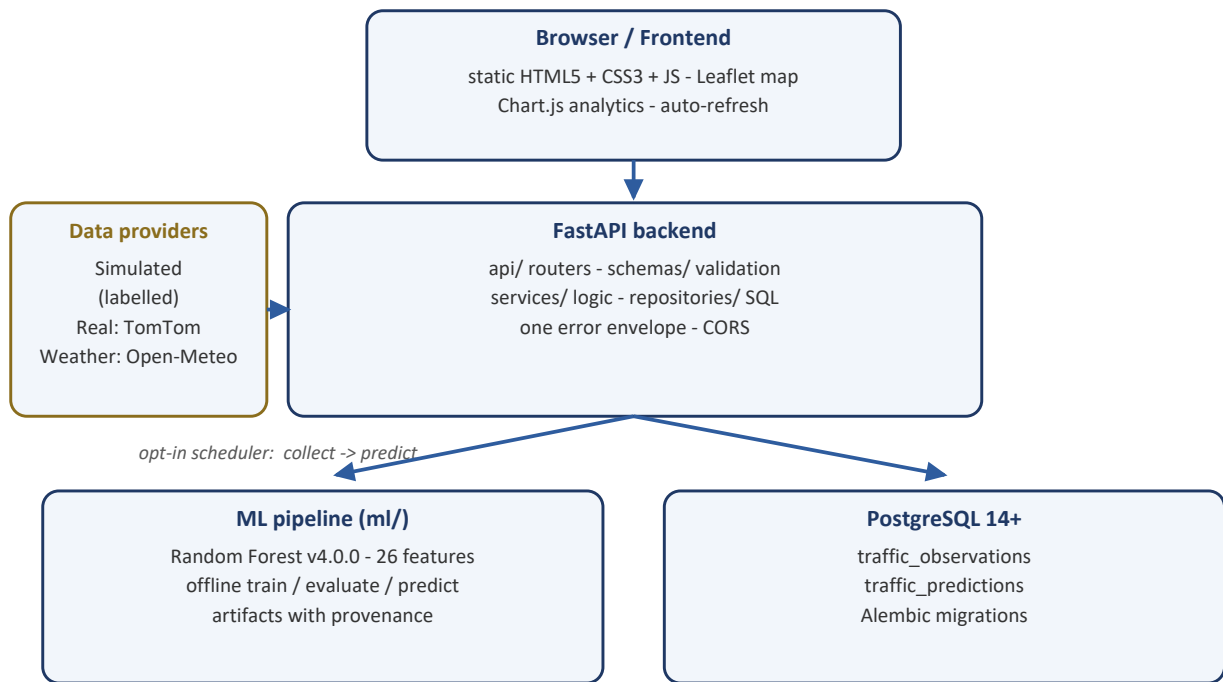


Figure 1: System architecture (data providers -> collector -> FastAPI -> dashboard; model scored offline by the independent ml/ pipeline).

Technologies Used

Layer	Technology	Notes
Frontend	HTML5, CSS3, Vanilla JavaScript (ES2020)	No build step; Leaflet 1.9.4 + Chart.js 4.4.1 vendored locally
Backend	Python 3.11+, FastAPI, Pydantic v2, Uvicorn	Application-factory pattern; typed schemas
Database	PostgreSQL 14+, SQLAlchemy 2.x, psycogp2	Env-driven lazy engine; Alembic migrations
ML	NumPy, pandas, scikit-learn, joblib	Independent package; artifact bundles with provenance
API client	httpx	Also drives mocked provider contract tests
Testing	pytest, httpx.MockTransport	451 tests; isolated SQLite; jsdom browser simulation

Data Collection

Observations are collected through a single provider interface (app/services/providers.py) so the storage, prediction and dashboard layers never depend on a vendor:

- **SimulatedTrafficProvider** - deterministic, seeded synthetic development data. Every record is stamped source='simulation' and every report, response and dashboard states it is not real traffic.
- **RealTrafficProvider** - live TomTom Flow Segment Data per monitored road: current and free-flow speed, confidence and road closure. A missing traffic API key fails with provider_not_configured rather than falling back to synthetic data.

- Weather - Open-Meteo forecast API alongside traffic (no key required in the free tier).
- Monitored roads - config/monitored_locations.json maps each coordinate to one of the six ML road segments (SEG-01..06); unknown segments are rejected on load.

Two data facts are deliberately honest: vehicle_count is NULL on live readings because no self-serve traffic API publishes vehicle throughput, and there is no historical traffic feed - TomTom reports current conditions only - so history is accumulated by the scheduler or backfilled (POST /api/traffic/collect/history).

Data Preprocessing

The six-stage offline pipeline (ml/preprocessing.py) runs independently of the API:

Stage	Function
Load	Reads CSV or Parquet, records SHA-256, returns a copy
Validate	Schema, dtypes, missing values, duplicates, timestamp parseability, physical ranges
Clean	Resolves duplicate keys, nulls impossible values, imputes per segment; refuses to drop more than 20% of rows
Features	Calendar fields, speed_ratio, strictly backward lags and rolling means, one-hot / frequency encoding
Target	Preserves a supplied label, otherwise derives four congestion bands from speed_ratio
Output	Writes traffic_processed.csv plus a .meta.json sidecar and data_summary.md

Required dataset columns: timestamp, road_segment_id, avg_speed_kph, plus free_flow_speed_kph or speed_ratio. Optional when present: flow_veh_per_hr, occupancy_pct, temperature_c, precipitation_mm, weather_condition, is_incident.

Feature Engineering

The feature set (26 features in model v4.0.0) is built to be leakage-safe:

- Calendar features (hour, weekday and similar) to capture peak-hour variation.
- Static segment capacity and encoded segment and weather identity.
- Strictly backward lag and rolling-mean features over the same road segment - rolling windows are shifted so the current reading is never included.
- All imputation is per segment, so no segment inherits another's values.
- Contemporaneous measurements (avg_speed_kph, flow_veh_per_hr, occupancy_pct, speed_ratio) are excluded because they determine the label; supplying them to training or to the API is rejected.

The congestion target is derived from $\text{speed_ratio} = \text{avg_speed_kph} / \text{free_flow_speed_kph}$ using left-closed bands: free_flow [0.75, oo), moderate [0.40, 0.75), heavy [0.15, 0.40), severe (-oo, 0.15). This is a documented convention, not measured ground truth.

Machine Learning Model

The model is a RandomForestClassifier, version 4.0.0 (26 features, four classes: free_flow, moderate, heavy, severe, label mapping 0-3). It is trained on a chronological 80/20 split - never a random one, because adjacent traffic observations are strongly autocorrelated - and evaluated with macro F1 as the headline metric so the rare 'severe' class is not ignored.

Property	Value
Estimator	RandomForestClassifier
Model version	4.0.0

Property	Value
Features	26
Classes	free_flow moderate heavy severe
min_history_per_segment	6
Training data	Simulated/development fixture (dataset_is_simulated=true)
Validated on real traffic	null / not performed
Evaluation	macro F1 as primary metric, against majority-class baseline

Training writes reproducible artifacts: traffic_model.pkl, preprocessor.pkl (feature order and label map - removing the train/serve skew failure mode), model_metadata.json (hyperparameters, split boundary, class balance, provenance), feature_importance.json, evaluation_results.json, model_report.md and confusion_matrix.png.

Prediction Workflow

A forecast always reads a segment's stored history; a caller cannot manufacture the lag features a prediction depends on:

- POST /api/traffic/predict stores the reading first, scores it against the stored prior observations for the same road_segment_id, persists the forecast, and returns both.
- The model needs at least min_history_per_segment (6) prior readings; otherwise the request answers 422 insufficient_history naming how many readings exist and how many are required - never a fabricated number.
- Collection runs the same path automatically: POST /api/traffic/collect stores each reading first, then scores it and reports one outcome per reading (predicted, insufficient_history, model_unavailable, unscoreable_location, failed). A prediction failure can never cost an observation.
- history/latest: GET /api/traffic/predictions and /predictions/latest return the stored forecasts joined to the observations they describe.
- The dashboard model card reports dataset_is_simulated and validated_on_real_traffic next to any forecast it renders.

Real-Time Traffic Pipeline

Live collection runs on an opt-in background scheduler (COLLECTION_ENABLED=true) or on a manual POST. Each cycle fetches current conditions per monitored location from TomTom Flow Segment Data and weather from Open-Meteo, normalises vendor fields onto the model's exact schema, stores with provenance, and auto-scores each new reading.

- Upstream throttle: TRAFFIC_MIN_REQUEST_INTERVAL and TRAFFIC_MAX_LOCATIONS keep one cycle well inside TomTom's documented quota.
- Failure classification: 429 -> 503 provider_rate_limited; other 4xx/5xx -> 502; timeout -> 504; an unusable body -> 502 provider_invalid_response. Vendor bodies are never echoed (they contain the request URL and the API key).
- A failed collection cycle is counted in /api/dashboard/status, logged, and never fatal.
- Freshness: measured from stored observation timestamps against DATA_FRESHNESS_THRESHOLD_SECONDS and reported fresh / stale / unavailable.
- Data provenance: every stored row carries a source label (simulation, tomtom, or an API caller's value) and the collector status endpoint reports whether the active provider is serving synthetic data.

Weather Integration

The model requires weather, and no traffic vendor publishes it, so Open-Meteo is used: a keyless forecast API with a documented free allowance. The client requests only the fields the model consumes - temperature_2m, precipitation and weather_code - and maps WMO codes onto the model's three condition labels (clear / cloudy / rain); an out-of-vocabulary code logs a warning and resolves to the nearest safe label rather than failing a collection. timezone=GMT keeps stored times comparable with the UTC timestamps on observations (no half-day shift for a Chennai deployment). WEATHER_ENABLED can switch the request off, with the caveat that readings stored without weather cannot be scored.

Verification: a live one-off call to the Open-Meteo forecast endpoint returned HTTP 200 with current.temperature_2m, current.precipitation and current.weather_code present - exactly the fields the client reads.

Backend API

FastAPI exposes a documented REST API (OpenAPI at /docs) with one error envelope {error: {code, message, details}}:

Method	Path	Purpose
GET	/api/health	Liveness probe
POST / GET	/api/traffic/observations[/id]	Record / list / fetch readings
GET	/api/traffic/latest	Most recent reading
POST	/api/traffic/predict	Store a reading, predict it, persist the forecast
POST	/api/traffic/observations/{id}/predict	Predict from a stored reading
GET	/api/traffic/predictions[/latest][status]	History, latest forecast, integration status
GET	/api/traffic/collector/status	Active provider + simulated flag
POST	/api/traffic/collect[/history]	Collect -> auto-predict / backfill history
GET	/api/dashboard/current trends locations status model	Single-poll dashboard reads

Health vs readiness: GET /api/health is a liveness probe; GET /api/dashboard/status reports readiness of each dependency (database, provider, scheduler, model, freshness). Dashboard reads are single-poll aggregations with a fixed query count (window functions, never N+1).

Frontend Dashboard

frontend/ ships a landing page and a dashboard (dashboard.html) with a sidebar (Overview, Live Traffic, Predictions, Analytics, Locations, System Status) and a location selector:

- Overview - six KPI cards, a Leaflet map with one marker per monitored location coloured by the predicted congestion class, and a per-location detail card.
- Live Traffic - current reading, forecast and freshness per location from /api/dashboard/current.
- Predictions - latest stored forecast and history.
- Analytics - Chart.js line chart (speed, free-flow and predicted class per point) for 1/6/12/24-hour windows from /api/dashboard/trends, plus the model card.
- Locations - readiness cards (e.g. '5 of 6 readings', stale/fresh, provider).
- System Status - database, providers, scheduler, model and freshness.

- Auto-refresh 15/30/60 s or manual, paused when the tab is hidden; every panel has loading, error and honest empty states; missing values render as N/A, never a placeholder number.

Leaflet and Chart.js are vendored locally (no CDN), and every API-derived string is HTML-escaped (UI.esc) before insertion - pinned by contract tests.

Database Design

Table	Purpose	Key constraints
traffic_observations	One reading per road per time	Unique (location_id, timestamp); avg_speed_kph <= free_flow_speed_kph; nullable vehicle_count; source provenance
traffic_predictions	One forecast per observation	FK ON DELETE CASCADE; predicted_congestion_level in 0-3; confidence nullable

Indexes cover the dashboard read paths: timestamp/location/road-name/segment on observations, and (observation_id, prediction_timestamp) plus observation/created on predictions. The schema is managed by a three-revision Alembic chain (table creation, nullable vehicle_count, history index). The DSN is resolved from application settings at runtime so no credential is ever committed to alembic.ini, and Alembic refuses to run without a target database rather than guessing one.

Testing

The suite runs with no external services: database tests use an isolated in-memory SQLite database, provider tests drive httpx.MockTransport, and the migrations are verified by rendering their SQL for PostgreSQL offline.

Item	Count / coverage
Backend API / domain tests	~422 tests at Stage 7 close
Stage 7 frontend contract tests	15 tests (structure, element contract, no secrets)
Stage 8 additions	7 production-guard config tests + 6 frontend XSS/escaping tests
Total (final run)	451 tests, all passing (plus 2 sklearn warnings)
Compile checks	python -m compileall -q (backend, ml, migrations, tests); node --check on every frontend/js/*.js

Coverage spans: the health contract, error envelope, configuration (including the production guard), DB wiring, schemas, provider response mappings, locations file, background scheduler, the full observation -> predict -> persist flow against the real trained artifact, auto-prediction failure modes, every dashboard endpoint with freshness behaviour, Alembic migrations, the ML pipeline, and the Stage 7/8 frontend contract and XSS/escaping tests.

Results

Result	Value
Automated tests	451 passed (0 failures)
Model version	4.0.0 (RandomForestClassifier, 26 features)
Train / test split	Chronological 80/20 (leakage-safe)
Macro F1 (simulated fixture)	0.7370
Accuracy (simulated fixture)	0.8561
Majority-class baseline	F1 0.2143 / accuracy 0.7500
Live weather verification	Open-Meteo HTTP 200
Migration verification	SQLite round-trip + offline PostgreSQL DDL

Verification detail: the full suite passed 451/451 on the final run (2 benign sklearn warnings). The Alembic round-trip on a fresh database reached head c3d4e5f6a7b8 with 2 tables, 12 indexes and the unique (location_id, timestamp) constraint, then downgraded and re-applied cleanly. Open-Meteo returned HTTP 200 live. TomTom live verification was not possible without a TRAFFIC_API_KEY and is covered by mock-transport contract tests instead.

Potential Benefits and Drawbacks

The problem statement asks specifically what such a system offers and what it costs. Benefits are stated first, then the drawbacks a responsible deployment must design against.

Potential benefits

- Forward-looking, not just descriptive: forecasts answer the question that can change driver behaviour - reroute before congestion forms, not after they are inside it.
- Measurable efficiency: fewer vehicles idling in avoidable congestion means less lost travel time, fuel burn and emissions - the standard, quantifiable outcomes in travel-time and emissions models.
- Reproducible and inspectable: a model replaces arbitrary colour-band thresholds; features, split, provenance and metrics are recorded beside every forecast instead of hidden behind a rule.
- Vendor-independent and retraining-ready: one provider interface means the storage, prediction and dashboard layers never change when a vendor does, and the pipeline is ready for a real dataset.
- Operationally safe: a production configuration guard, one error envelope, classified upstream failures and an opt-in scheduler.
- Cheap to pilot: free-tier sources (TomTom's documented allowance, keyless Open-Meteo) and open-source tooling keep a small institutional or municipal pilot low-cost.
- Verifiable: 451 automated tests, compile checks, and documented live-endpoint verification where credentials permitted.

Potential drawbacks

- Data gaps limit trust: live feeds publish current conditions only and are quota-limited, and there is no historical or throughput feed - a fresh location needs six prior readings before the first forecast, and vehicle throughput stays unknown.
- Unvalidated model risk: the model is trained on simulated data, so forecasts are demonstrative until retrained and validated against real ground truth; every artifact labels dataset_is_simulated.
- Privacy and liability: deriving and storing travel behaviour raises privacy concerns, and a publicly served wrong forecast (for example near an emergency) could carry safety or legal consequences.

- External dependency: the service depends on vendor uptime, quota and the weather API; operator error on the provider/key pair could surface labelled simulation as real without the production guard.
- Equity: rerouting nudges and infrastructure spend can concentrate on busy corridors; without checks the benefits of prediction need not reach every neighbourhood equally.
- False confidence: a polished dashboard reads as authoritative, so honest provenance and validation status are embedded in the product rather than added later.
- The API is unauthenticated by design and the live PostgreSQL round-trip was not executed here; gateway authentication, rate limiting and a hosted PostgreSQL are required before a fully public, production-grade launch.

Future Enhancements

- Retrain and revalidate on a real urban traffic dataset once sufficient history is collected (the pipeline is ready and tested).
- Delivered in this session: a container image (backend/Dockerfile), docker-compose (PostgreSQL + API), a CI workflow running the suite and image build, and a single-URL public deployment (render.yaml).
- Gateway authentication and rate limiting for public exposure.
- A historical or throughput data source should one become self-serve.
- A staging environment, plus a live PostgreSQL round-trip in CI.

Deployment

Deployment is environment-driven and guard-railed. Copy `.env.example` to `.env`, set `DATABASE_URL`, apply the schema with alembic upgrade head, then start from the project root so the ml package is importable:

```
.\venv\Scripts\python.exe -m uvicorn app.main:app --app-dir backend --host 0.0.0.0 --port 8000
```

When `ENVIRONMENT=production` the application refuses to start unless `TRAFFIC_PROVIDER=real`, `TRAFFIC_API_KEY` and a PostgreSQL `DATABASE_URL` are all set - a missing piece fails at boot naming the setting, never echoing the secret, so simulated data can never be served in production. `DB_FAIL_FAST=true` makes an unreachable database fail the same way. Liveness (`/api/health`) and readiness (`/api/dashboard/status`) are distinct, and guidance covers quota maths, one-worker operation, reverse-proxy and static-frontend serving and a troubleshooting table. Full detail: [docs/deployment.md](#).

Deployment was exercised rather than only documented. The same repository ships a Docker image (backend/Dockerfile) that serves both the API and the static dashboard from one origin (`FRONTEND_DIR`), a docker-compose topology (PostgreSQL 14 + API) for one-command local runs, and a GitHub Actions CI (full pytest suite + image build) that is green on the default branch. A `render.yaml` Blueprint provisions a free Render Web Service that migrates the schema at boot and starts Uvicorn on Render's injected `PORT` with `COLLECTION_ENABLED=true` and `TRAFFIC_PROVIDER=real`, so the hosted dashboard accumulates live TomTom readings and scores them automatically.

Conclusion

The project delivers a complete, modular AI-based urban traffic congestion prediction system: a leakage-safe data pipeline, an evaluated Random Forest classifier, real-time ingestion behind a provider interface, an automatic prediction engine, a dashboard API and interactive frontend, and a hardened production configuration. It is verified by 451 automated tests and documented end to end.

The decisive caveat is stated rather than hidden: the training data is a simulated fixture, so current ML forecasts are demonstrative. The same, tested pipeline is ready to be retrained and validated against real-world ground truth as soon as sufficient real traffic observations are collected. Until then, this is a working, honest pipeline - not a field-validated traffic predictor.

Appendix

A. Commands used and verified

```
python -m pip install -r requirements.txt
Copy-Item .env.example .env # then set DATABASE_URL, provider
alembic upgrade head # apply schema; --sql to review offline
.\.venv\Scripts\python.exe -m uvicorn app.main:app --app-dir backend
pytest -q # 451 passed
python -m compileall -q backend ml migrations tests
node --check frontend/js/<file>; (repeat for every file)
python -m http.server 5500 --directory frontend # serve the dashboard
```

B. Migration verification detail

Item	Method	Result
Migration round-trip	alembic upgrade head -> downgrade base -> upgrade head (fresh SQLite)	Reach head c3d4e5f6a7b8; 2 tables, 12 indexes, unique constraint; re-applied cleanly
PostgreSQL dialect	Offline --sql rendering + migration tests	Dialect-correct DDL verified
Live PostgreSQL round-trip	Not possible here (no server)	Procedure documented; plain alembic upgrade head

C. Repository layout (summary)

```
backend/app/ FastAPI application (api, schemas, services, repositories, database, core)
ml/ ML package (preprocessing, train, evaluate, predict, data, models)
frontend/ static dashboard (html, css, js, vendor/Leaflet + Chart.js)
migrations/ Alembic environment + 3 versions
tests/ pytest suite
docs/ architecture, api, ml-pipeline, roadmap, deployment, report, presentation
```

D. Honest-use statement

All metrics in this report describe the simulated development fixture. The model has not been validated on real-world ground truth; any presentation of a forecast as a field-tested number would be false.